

## 03\_dependencias-y-requirements

#python #fastapi #pydantic #SQLAlchemy #PostgreSQL #alembic #pytest #docker #backend #SIEM

### 1 Objetivo de la nota

Esta nota analiza las dependencias y archivos de configuración principales del backend del laboratorio SIEM MVP.

El objetivo es entender qué librerías necesita el backend, cómo se instalan, cómo se configura pytest, cómo se construye la imagen Docker del backend y cómo Alembic obtiene la URL de conexión a PostgreSQL.

Los archivos analizados son:

```
backend/requirements.txt
backend/pytest.ini
backend/Dockerfile
backend/alembic.ini
```

### 2 Comandos utilizados

Para visualizar los archivos se ejecutaron estos comandos desde la raíz del proyecto:

```
cd ~/siem-lab

cat backend/requirements.txt
cat backend/pytest.ini
sed -n '1,220p' backend/Dockerfile
sed -n '1,220p' backend/alembic.ini
```

Estos comandos permiten revisar:

```
requirements.txt
↓
dependencias Python

pytest.ini
↓
configuración de tests

Dockerfile
↓
construcción del contenedor backend

alembic.ini
↓
configuración de migraciones
```

### 3 Archivo backend/requirements.txt

El contenido del archivo es:

```
fastapi
uvicorn
sqlalchemy>=2.0
psycopg[binary]
alembic
pytest
httpx
```

Este archivo define las dependencias Python necesarias para que el backend funcione.

Su papel es permitir reconstruir el entorno sin entregar el directorio `venv/`.

El flujo habitual sería:

```
cd backend
python3 -m venv venv
source venv/bin/activate
pip install -r requirements.txt
```

En Docker, este archivo también se utiliza para instalar las dependencias dentro del contenedor.

---

## 4 Dependencia fastapi

```
fastapi
```

FastAPI es el framework principal del backend.

Permite crear la API REST del laboratorio.

En el proyecto se usa para:

- Crear la aplicación principal.
- Definir endpoints HTTP.
- Registrar routers.
- Usar Depends para inyección de dependencias.
- Devolver respuestas JSON.
- Generar documentación Swagger.

Ejemplo de uso dentro del proyecto:

```
from fastapi import FastAPI

app = FastAPI(title="SIEM Backend", version="0.1.0")
```

FastAPI es la base sobre la que se construyen endpoints como:

```
/health
/info
/events
/ingest
/rules
/alerts
/metrics
```

---

## 5 Dependencia uvicorn

```
uvicorn
```

Uvicorn es el servidor ASGI que ejecuta la aplicación FastAPI.

FastAPI define la aplicación, pero Uvicorn es quien la sirve por HTTP.

En el Dockerfile se usa así:

```
CMD ["uvicorn", "app.main:app", "--reload", "--host", "0.0.0.0", "--port", "8000"]
```

Esto significa:

```
uvicorn
  ↓
ejecuta app.main:app

--host 0.0.0.0
```

```
↓
permite recibir conexiones dentro del contenedor

--port 8000
↓
expone la API en el puerto 8000

--reload
↓
recarga automática en desarrollo
```

---

## 6 Dependencia sqlalchemy>=2.0

```
sqlalchemy>=2.0
```

SQLAlchemy es la librería ORM utilizada para trabajar con PostgreSQL desde Python.

El requisito indica que debe instalarse la versión 2.0 o superior.

En el proyecto se usa para:

- Crear el engine de base de datos.
- Crear sesiones.
- Definir modelos ORM.
- Construir consultas.
- Insertar, consultar y actualizar datos.

Ejemplos de uso:

```
from sqlalchemy import create_engine
from sqlalchemy.orm import sessionmaker
```

También se usa en modelos como:

```
from sqlalchemy.orm import Mapped, mapped_column
```

Y en consultas como:

```
select(Alert).order_by(Alert.created_at.desc())
```

SQLAlchemy conecta la lógica Python con tablas como:

```
events
rules
alerts
```

---

## 7 Dependencia psycopg[binary]

```
psycopg[binary]
```

psycopg es el driver que permite a Python conectarse a PostgreSQL.

SQLAlchemy necesita un driver real para hablar con la base de datos.

En este proyecto, la URL de conexión tiene este formato:

```
DATABASE_URL=postgresql+psycopg://siem:change_me@db:5432/siem
```

La parte importante es:

```
postgresql+psycopg
```

Esto indica que SQLAlchemy debe usar `psycopg` como driver PostgreSQL.

El extra `[binary]` facilita la instalación porque incluye componentes precompilados.

---

## 8 Dependencia `alembic`

```
alembic
```

Alembic es la herramienta de migraciones de base de datos.

Se utiliza para versionar los cambios en la estructura de PostgreSQL.

En el proyecto aparece la carpeta:

```
backend/alembic/
```

y el archivo:

```
backend/alembic.ini
```

Las migraciones permiten reflejar cambios como:

- creación de tablas
- añadir columnas
- modificar defaults
- añadir campos como `meta`, `group_key`, `throttle`, `threshold`, `status` o `updated_at`

Relación:

```
Modelos SQLAlchemy
      ↓
Migraciones Alembic
      ↓
Cambios en PostgreSQL
```

---

## 9 Dependencia `pytest`

```
pytest
```

Pytest es la herramienta de testing del backend.

Permite ejecutar pruebas automatizadas.

En el proyecto existen:

```
backend/tests/test_health.py
backend/tests/test_alerts_ui.py
backend/pytest.ini
```

Esto indica que el backend tiene pruebas preparadas para validar partes del sistema.

Comando habitual:

```
cd backend
pytest
```

---

## 10 Dependencia `httpx`

```
httpx
```

HTTPX es una librería cliente HTTP.

En proyectos FastAPI se usa habitualmente para pruebas de endpoints, especialmente junto con `TestClient`.

Aunque no se ha analizado todavía el contenido de los tests, su presencia en `requirements.txt` encaja con pruebas automatizadas de API.

Uso conceptual:

```
pytest
  ↓
cliente de pruebas
  ↓
peticiones HTTP internas
  ↓
endpoints FastAPI
```

---

## 1.1 Resumen de dependencias

Las dependencias pueden agruparse así:

```
API
├─ fastapi
└─ uvicorn

Base de datos
├─ sqlalchemy>=2.0
└─ psycopg[binary]

Migraciones
└─ alembic

Testing
├─ pytest
└─ httpx
```

Relación funcional:

```
FastAPI
  ↓
expone endpoints

Uvicorn
  ↓
sirve la aplicación

SQLAlchemy + psycopg
  ↓
conectan con PostgreSQL

Alembic
  ↓
gestiona migraciones

pytest + httpx
  ↓
permiten pruebas automatizadas
```

---

## 1.2 Archivo `backend/pytest.ini`

El contenido es:

```
[pytest]
pythonpath = .
testpaths = tests
```

Este archivo configura cómo debe ejecutar pytest las pruebas del backend.

---

## 1.3 Sección [pytest]

```
[pytest]
```

Indica que las opciones siguientes pertenecen a pytest.

Es la sección principal de configuración.

---

## 1.4 Configuración pythonpath = .

```
pythonpath = .
```

Esta línea añade el directorio actual al path de Python durante la ejecución de tests.

Esto permite que los tests puedan importar módulos del proyecto.

Por ejemplo, permite importaciones como:

```
from app.main import app
```

sin tener que instalar el paquete de forma explícita.

El punto:

```
.
```

representa el directorio actual.

Como los tests se ejecutan desde `backend/`, el path base será:

```
backend/
```

---

## 1.5 Configuración testpaths = tests

```
testpaths = tests
```

Indica que pytest debe buscar pruebas dentro de la carpeta:

```
backend/tests/
```

Esto evita que pytest busque tests en otras partes del proyecto.

La relación es:

```
pytest
  ↓
lee pytest.ini
  ↓
busca en tests/
  ↓
ejecuta test_health.py y test_alerts_ui.py
```

---

## 16 Archivo backend/Dockerfile

El contenido es:

```
FROM python:3.12-slim

WORKDIR /app

COPY requirements.txt /app/requirements.txt
RUN pip install --no-cache-dir -r requirements.txt

COPY . /app

CMD ["uvicorn", "app.main:app", "--reload", "--host", "0.0.0.0", "--port", "8000"]
```

Este archivo define cómo construir el contenedor Docker del backend.

Docker permite ejecutar el backend en un entorno controlado, sin depender directamente del Python instalado en la máquina.

## 17 Imagen base

```
FROM python:3.12-slim
```

Indica que la imagen del backend parte de una imagen oficial de Python.

Desglose:

```
python
  ↓
imagen oficial con Python

3.12
  ↓
versión de Python utilizada

slim
  ↓
versión reducida de la imagen
```

Usar `slim` reduce el tamaño de la imagen.

## 18 Directorio de trabajo

```
WORKDIR /app
```

Define el directorio de trabajo dentro del contenedor.

A partir de esta línea, los comandos se ejecutan dentro de:

```
/app
```

Esto significa que el código del backend vivirá dentro del contenedor en esa ruta.

## 19 Copia de requirements.txt

```
COPY requirements.txt /app/requirements.txt
```

Copia el archivo de dependencias desde el backend local al contenedor.

Origen:

```
backend/requirements.txt
```

Destino:

```
/app/requirements.txt
```

Se copia primero para instalar dependencias antes de copiar todo el código.

Esto es una práctica habitual en Docker porque mejora el uso de caché.

---

## 2.0 Instalación de dependencias

```
RUN pip install --no-cache-dir -r requirements.txt
```

Instala las dependencias Python dentro del contenedor.

Desglose:

```
pip install
  ↓
instala paquetes Python

--no-cache-dir
  ↓
evita guardar caché de pip dentro de la imagen

-r requirements.txt
  ↓
instala lo indicado en requirements.txt
```

Esto instala:

```
fastapi
uvicorn
sqlalchemy
psycopg2
alembic
pytest
httpx
```

---

## 2.1 Copia del código

```
COPY . /app
```

Copia todo el contenido del directorio backend dentro del contenedor.

Origen:

```
backend/
```

Destino:

```
/app
```

Esto incluye:

```
app/
alembic/
alembic.ini
requirements.txt
```



```
pytest.ini
tests/
```

Punto importante: es importante que el `.dockerignore`, si existiera, excluyera elementos como `venv/`, `__pycache__/` y `.pytest_cache/`.

Si no existe `.dockerignore`, Docker puede copiar más de lo necesario. En la entrega, aun así, el ZIP ya se preparó excluyendo basura técnica.

## 2.2 Comando de arranque

```
CMD ["uvicorn", "app.main:app", "--reload", "--host", "0.0.0.0", "--port", "8000"]
```

Define el comando que se ejecutará cuando arranque el contenedor.

Desglose:

```
uvicorn
  ↓
servidor ASGI

app.main:app
  ↓
archivo app/main.py, variable app

--reload
  ↓
recarga automática en desarrollo

--host 0.0.0.0
  ↓
escucha en todas las interfaces del contenedor

--port 8000
  ↓
puerto interno de la API
```

La API quedará disponible dentro del contenedor en:

```
0.0.0.0:8000
```

Y Docker Compose la publicará hacia el host según la configuración del servicio.

## 2.3 Punto importante sobre `--reload`

El uso de:

```
--reload
```

es cómodo en desarrollo porque reinicia el servidor cuando cambia el código.

Para un entorno de producción, normalmente se eliminaría.

En este proyecto tiene sentido porque se trata de un laboratorio MVP académico.

## 2.4 Archivo `backend/alembic.ini`

El archivo `alembic.ini` configura Alembic.

Contenido principal:

```
# A generic, single database configuration.

[alembic]
script_location = %(here)s/alembic
prepend_sys_path = .
path_separator = os

# IMPORTANTE:
# No hardcodeamos credenciales aquí.
# Alembic leerá la URL desde la variable de entorno DATABASE_URL.
sqlalchemy.url = %(DATABASE_URL)s
```

Después incluye secciones de hooks y logging.

---

## 2.5 Sección [alembic]

```
[alembic]
```

Define la configuración principal de Alembic.

---

## 2.6 Ubicación de scripts

```
script_location = %(here)s/alembic
```

Indica dónde están los scripts de migración.

`%(here)s` representa el directorio donde está el archivo `alembic.ini`.

Como `alembic.ini` está en:

```
backend/alembic.ini
```

la carpeta de migraciones queda en:

```
backend/alembic/
```

---

## 2.7 prepend\_sys\_path = .

```
prepend_sys_path = .
```

Añade el directorio actual al path de Python cuando Alembic se ejecuta.

Esto ayuda a que Alembic pueda importar módulos de la aplicación.

Por ejemplo:

```
app.models
app.db
```

---

## 2.8 Separador de rutas

```
path_separator = os
```

Indica que Alembic use el separador de rutas propio del sistema operativo.

En Linux, el separador principal de rutas es:

```
/
```

---

## 2.9 URL de SQLAlchemy

```
sqlalchemy.url = %(DATABASE_URL)s
```

Esta línea es especialmente importante.

No se escriben credenciales directamente en `alembic.ini`.

En lugar de eso, Alembic lee la URL desde la variable de entorno:

```
DATABASE_URL
```

Esto evita dejar credenciales fijas dentro del archivo de configuración.

La URL real o de ejemplo viene definida en:

```
DATABASE_URL=postgresql+psycopg://siem:change_me@db:5432/siem
```

Relación:

```
.env / docker env
    ↓
DATABASE_URL
    ↓
alembic.ini
    ↓
Alembic conecta con PostgreSQL
```

---

## 3.0 Importancia de no hardcodear credenciales

El propio archivo incluye el comentario:

```
# No hardcodeamos credenciales aquí.
# Alembic leerá la URL desde la variable de entorno DATABASE_URL.
```

Esto es una buena práctica.

Ventajas:

- Evita guardar credenciales reales en el repositorio.
- Permite usar distintas bases de datos según entorno.
- Facilita Docker.
- Facilita despliegues futuros.

---

## 3.1 Sección `[post_write_hooks]`

```
[post_write_hooks]
# hooks = ruff
# ruff.type = module
# ruff.module = ruff
# ruff.options = check --fix REVISION_SCRIPT_FILENAME
```

Esta sección está preparada para hooks posteriores a la creación de migraciones.

En este caso, están comentados.

Esto significa que no se ejecutan.

Podrían usarse en el futuro para formatear o revisar automáticamente scripts de migración con herramientas como Ruff.

---

## 3.2 Secciones de logging

El archivo también incluye configuración de logs:

```
[loggers]
keys = root,sqlalchemy,alembic

[handlers]
keys = console

[formatters]
keys = generic
```

Esto define cómo Alembic y SQLAlchemy muestran mensajes por consola.

---

## 3.3 Logger raíz

```
[logger_root]
level = WARNING
handlers = console
qualname =
```

Define el nivel global de logs como `WARNING`.

Esto evita mostrar demasiada información si no es necesaria.

---

## 3.4 Logger de SQLAlchemy

```
[logger_sqlalchemy]
level = WARNING
handlers =
qualname = sqlalchemy.engine
```

Configura los logs del motor SQLAlchemy.

Nivel:

```
WARNING
```

Esto evita que SQLAlchemy muestre cada consulta SQL salvo que haya advertencias.

---

## 3.5 Logger de Alembic

```
[logger_alembic]
level = INFO
handlers =
qualname = alembic
```

Configura los logs de Alembic en nivel `INFO`.

Esto permite ver información útil durante migraciones.

---

## 3.6 Handler de consola

```
[handler_console]
class = StreamHandler
args = (sys.stderr,)
level = NOTSET
formatter = generic
```

Define que los logs se muestren por consola, concretamente por `stderr`.

## 3.7 Formato de logs

```
[formatter_generic]
format = %(levelname)-5.5s [%(name)s] %(message)s
datefmt = %H:%M:%S
```

Define cómo se muestran los mensajes de log.

Formato conceptual:

```
LEVEL [nombre_logger] mensaje
```

Ejemplo:

```
INFO [alembic.runtime.migration] Running upgrade
```

## 3.8 Relación entre requirements, Dockerfile y Alembic

Estos archivos trabajan juntos.

Relación:

```
requirements.txt
  ↓
declara dependencias

Dockerfile
  ↓
instala dependencias dentro del contenedor

alembic.ini
  ↓
usa DATABASE_URL para migraciones

pytest.ini
  ↓
configura pruebas automatizadas
```

Flujo de ejecución:

```
Docker construye imagen
  ↓
pip install -r requirements.txt
  ↓
uvicorn ejecuta FastAPI
  ↓
FastAPI usa SQLAlchemy
  ↓
SQLAlchemy usa psycopg
  ↓
PostgreSQL almacena datos
```

Flujo de migración:

```
Alembic
  ↓
lee alembic.ini
  ↓
usa DATABASE_URL
```

```
↓
aplica scripts en alembic/versions/
↓
actualiza PostgreSQL
```

Flujo de pruebas:

```
pytest
↓
lee pytest.ini
↓
busca tests/
↓
usa httpx/TestClient
↓
valida endpoints
```

---

## 3 9 Archivos que deben entregarse

Estos archivos sí deben incluirse en la entrega:

```
backend/requirements.txt
backend/pytest.ini
backend/Dockerfile
backend/alembic.ini
backend/alembic/
```

Motivo:

```
requirements.txt
↓
permite instalar dependencias

pytest.ini
↓
permite ejecutar tests correctamente

Dockerfile
↓
permite construir el backend

alembic.ini
↓
configura migraciones

alembic/
↓
contiene scripts de migración
```

---

## 4 0 Archivos relacionados que no deben entregarse

No deben incluirse:

```
backend/venv/
backend/.pytest_cache/
backend/**/__pycache__/*
backend/**/*.*pyc
```

Motivo:

```
venv/  
  ↓  
se reconstruye con requirements.txt  
  
.pytest_cache/  
  ↓  
lo genera pytest  
  
__pycache__/ y *.pyc  
  ↓  
los genera Python
```

## 4.1 Comandos útiles

Instalar dependencias en entorno local:

```
cd ~/siem-lab/backend  
python3 -m venv venv  
source venv/bin/activate  
pip install -r requirements.txt
```

Ejecutar backend localmente:

```
uvicorn app.main:app --reload
```

Ejecutar tests:

```
cd ~/siem-lab/backend  
pytest
```

Ver configuración de pytest:

```
cat backend/pytest.ini
```

Ver dependencias:

```
cat backend/requirements.txt
```

Construir imagen Docker desde el backend:

```
cd ~/siem-lab/backend  
docker build -t siem-backend .
```

Ver configuración de Alembic:

```
sed -n '1,220p' backend/alembic.ini
```

## 4.2 Resumen técnico

El backend depende de un conjunto pequeño pero suficiente de librerías:

```
fastapi  
uvicorn  
sqlalchemy  
psycopg  
alembic  
pytest  
httpx
```

Estas dependencias cubren:

```
API
servidor ASGI
ORM
conexión PostgreSQL
migraciones
testing
```

El archivo `requirements.txt` permite reconstruir el entorno Python.

El archivo `pytest.ini` configura las pruebas.

El `Dockerfile` construye el contenedor del backend.

El archivo `alembic.ini` configura las migraciones sin hardcodear credenciales, usando `DATABASE_URL` .

En conjunto, estos archivos hacen que el backend sea ejecutable, mantenible, migrable y testeable.